Universitatea Națională de Științe și Tehnologii
POLITEHNICA București

PROIECT 2 – APLICAȚII PRACTICE ALE MICROCONTROLERELOR

ECHIPA NR. 5

Studenți :

Păun Alexandru-Nicolae
Petrescu Oana Ștefania
Popa Emilia
Șerbănoiu Ana-Maria

Grupa:

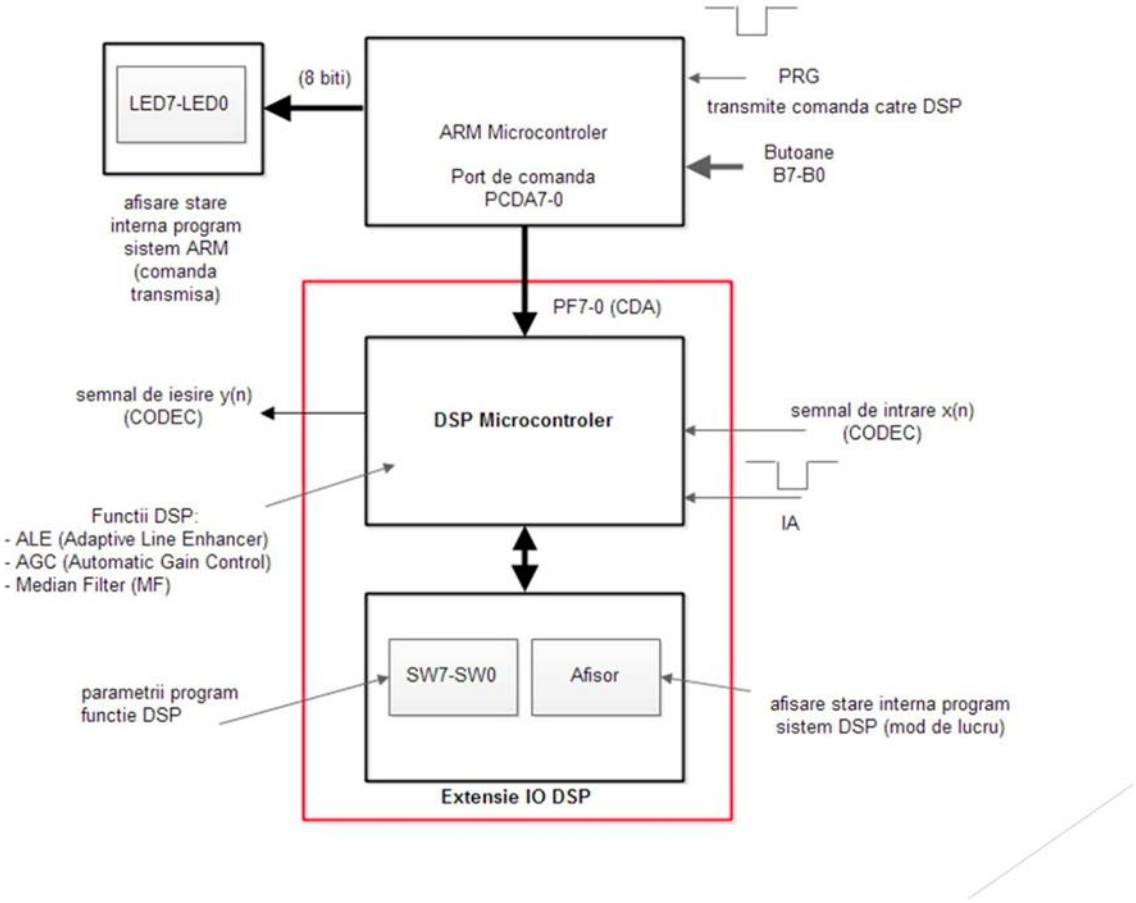
431D

Profesor coordonator : Zoican Sorin

1. Descrierea sistemului

Sistemul implementat este compus din două subsisteme interconectate — ARM și DSP, având rolul de a procesa semnalele. Subsistemul ARM preia comenzile utilizatorului și le transmite către DSP. Subsistemul DSP procesează semnalul aplicând diverse funcții specifice pentru a genera semnalul de ieșire.

2. Schema bloc a sistemului



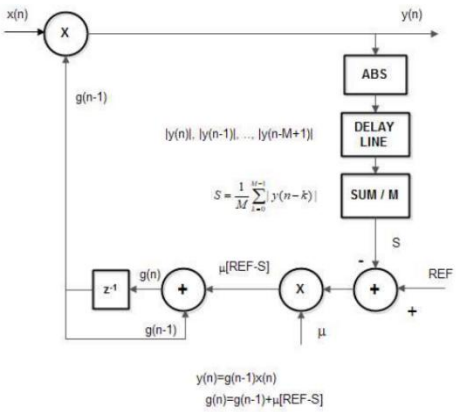
- **Microcontrolerul ARM (STM32F103CBTx):** citește butoanele apăsate de utilizator, construiește o comandă și o trimite către DSP prin pinii PCDA7-0. Pe LED-uri afișează ce comandă a trimis.
- **Microcontrolerul DSP (ADSP-2181 EZ-KIT Lite):** primește sunetul de la codec-ul AD1847, îl prelucrează în funcție de comanda primită de la ARM (AGC, ALE sau MF) și îl trimite înapoi la codec pentru redare.
- **Codec-ul AD1847 SoundPort:** convertește semnalul audio analog de intrare în format digital pe 16 biți și reconstituie semnalul de ieșire din digital în analog. Comunicarea cu DSP-ul se face prin portul serial SPORT0.

- **Extensia IO DSP:** conține comutatoarele SW7-SW0 prin care utilizatorul poate configura manual parametrii funcțiilor DSP, și un afișor care arată starea internă a sistemului DSP în timp real.

3. Descrierea funcțiilor de procesare

- **Automatic Gain Control (AGC)** - este o funcție de procesare a semnalului care menține automat volumul semnalului de ieșire la un nivel constant, indiferent de variațiile semnalului de intrare.

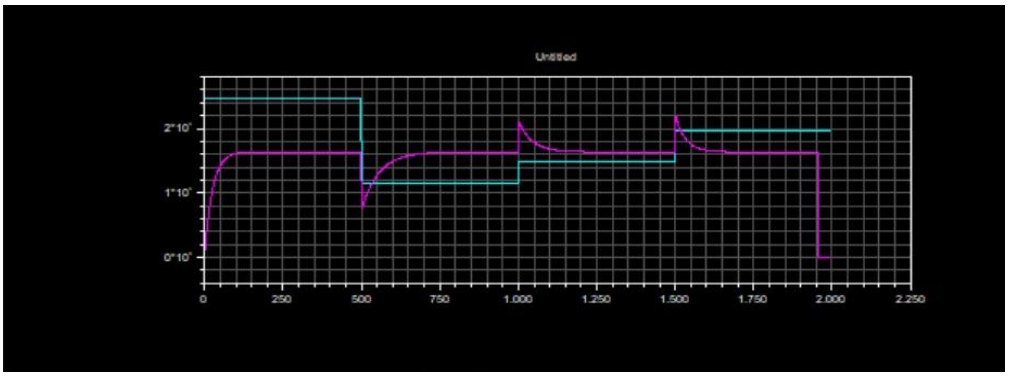
La fiecare eșantion, semnalul de intrare x(n) este înmulțit cu câștigul g(n-1) calculat anterior, obținând semnalul de ieșire y(n). Apoi se calculează media S a valorilor absolute ale ultimelor M eșantioane de ieșire. Câștigul se actualizează în funcție de diferența dintre nivelul de referință REF și media S — dacă semnalul e prea slab, câștigul crește, dacă e prea puternic, câștigul scade.



Parametrii funcției sunt:

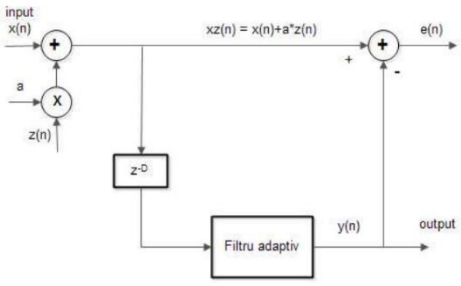
- **REF** — nivelul țintă al semnalului de ieșire
- **M** — numărul de eșantioane folosite pentru calculul mediei
- **μ** — viteza de adaptare a câștigului

Simularea in VisualDSP++



- **Adaptive Line Enhancer (ALE)** - este o funcție de procesare a semnalului care separă componentele periodice (tonuri, sinusoide) de zgomotul aleator dintr-un semnal mixt.

Semnalul de intrare x(n) este combinat cu o versiune întârziată z(n) înmulțită cu coeficientul a, obținând semnalul xz(n). Acesta este introdus într-un filtru adaptiv cu întârziere D, care calculează ieșirea y(n) ca o combinație liniară a eșantioanelor anterioare ponderate cu coeficienții h(k). Eroarea e(n) reprezintă diferența dintre xz(n) și y(n) și este folosită pentru actualizarea coeficienților filtrului — practic filtrul învață să recunoască și să reproducă componenta periodică a semnalului.

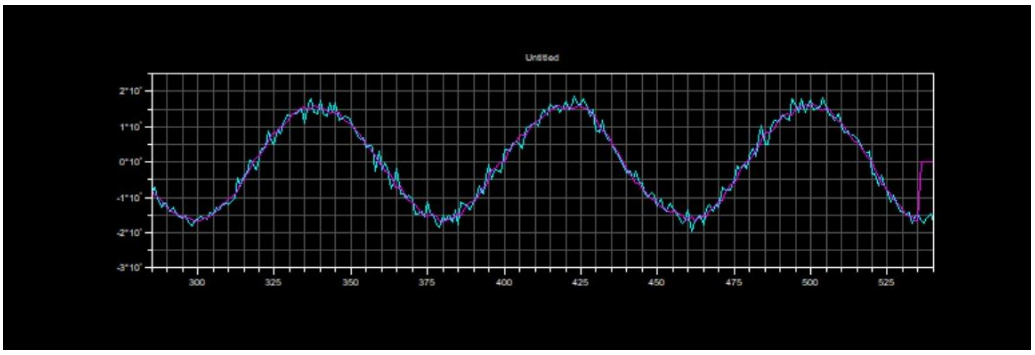


$$y(n) = \sum_{k=0}^{N-1} h(k).xz(n-k-D)$$
$$h(k) = (1-\mu.\lambda).h(k) + \mu.xz(n-k-D).e(n)\ k = 0,1,\ldots,N-1$$

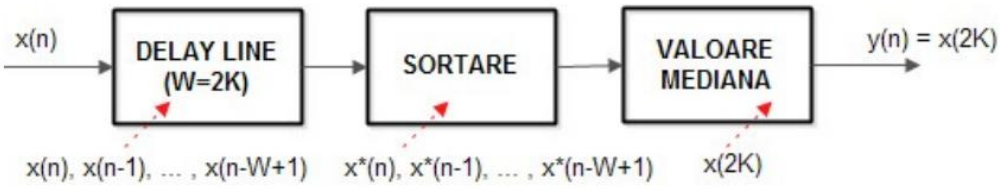
Parametrii funcției sunt:

- **k (N)** — lungimea filtrului adaptiv (numărul de coeficienți)
- **a** — coeficientul de combinare a semnalului întârziat
- **μ** — viteza de adaptare a coeficienților
- **λ** — factorul de uitare (controlează stabilitatea filtrului)

Simularea in VisualDSP++



- **Median Filter (MF)** - o funcție de procesare a semnalului care elimină zgomotul dintr-un semnal, păstrând în același timp forma generală a acestuia.

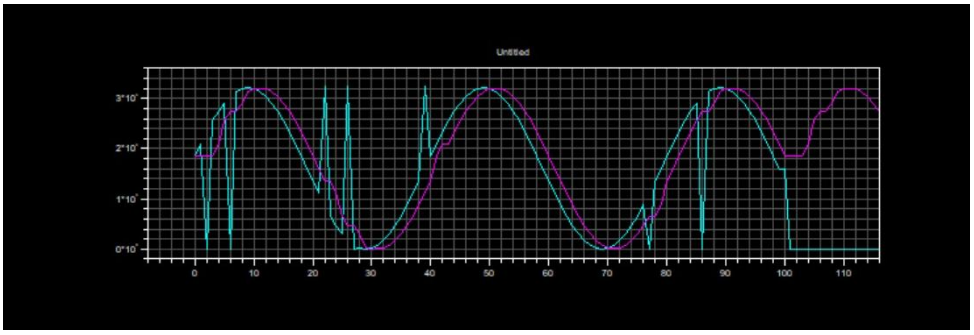


Se păstrează în memorie ultimele W=2K eșantioane ale semnalului de intrare într-o linie de întârziere (delay line). Cele 2K eșantioane sunt apoi sortate în ordine crescătoare. Din șirul sortat se selectează valoarea din mijloc care devine eșantionul de ieșire y(n).

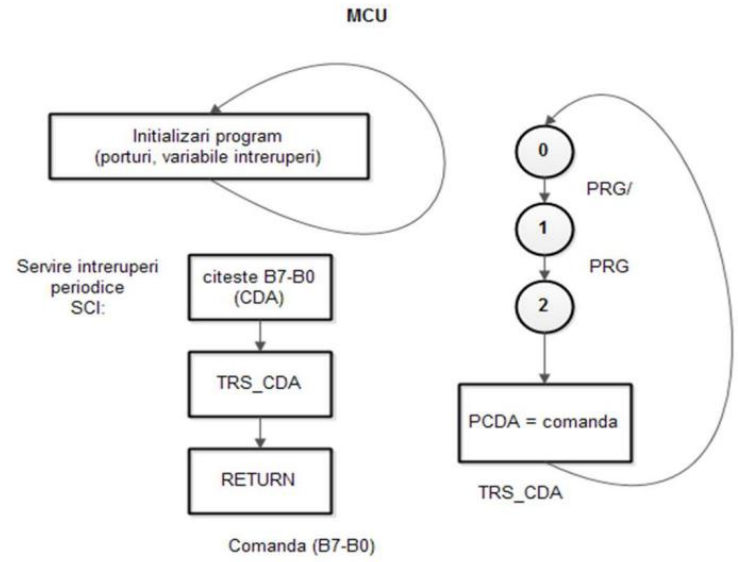
Parametrul funcției este:

- **K** — jumătate din dimensiunea ferestrei de sortare (W=2K eșantioane)

Simularea in VisualDSP++



4. ARM



D7	D6	D5	D4	D3	D2	D1	D0
1	1	CDA1	CDA0	CANAL	ID2	ID1	ID0

PRG - buton fara retinere

Microcontrolerul ARM are rolul de a citi butoanele utilizatorului și de a trimite o comandă către DSP.

La pornire se execută **inițializările** necesare — configurarea porturilor și a variabilelor de întreruperi — după care programul intră în bucla principală de așteptare.

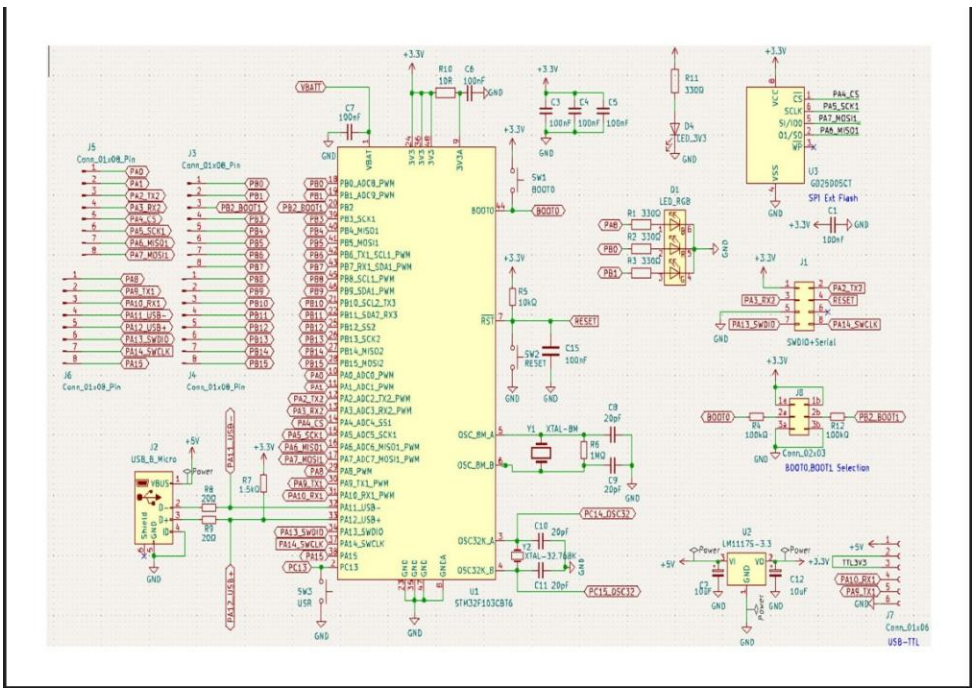
Întreruperea periodică (TIM2) implementează o mașină de stări cu 3 stări pentru gestionarea butonului PRG și transmiterea comenzii către DSP:

- **Starea 0** – programul așteaptă ca butonul PRG să fie eliberat (PRG/ = semnal negat, adică buton neapăsat). Când detectează că PRG nu este apăsat, trece în starea 1.
- **Starea 1** – programul așteaptă apăsarea butonului PRG. Când îl detectează apăsat, citește valorile butoanelor B7–B0 de pe portul CDA, construiește comanda și trece în starea 2.
- **Starea 2** – se scrie comanda pe portul PCDA (care merge fizic la DSP) și se apelează TRS_CDA pentru transmitere. După aceea se revine în starea 0.

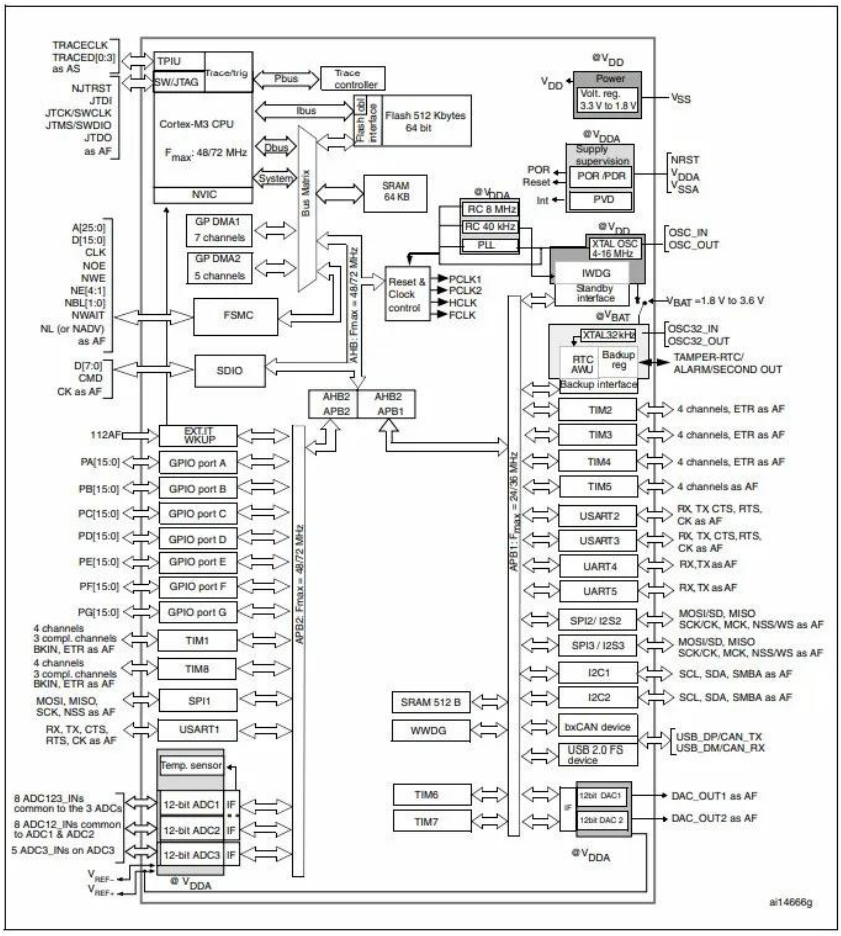
Structura comenzii (B7-B0):

- D7, D6 = mereu 1 — biți de validare a comenzii
- D5, D4 = CDA1, CDA0 — identifică parametrul transmis
- D3 = CANAL — selectează canalul audio (0=stânga, 1=dreapta)
- D2, D1, D0 = ID funcție (0=AGC, 1=ALE, 2=MF)

Schema electrică



Schema bloc STM32F103

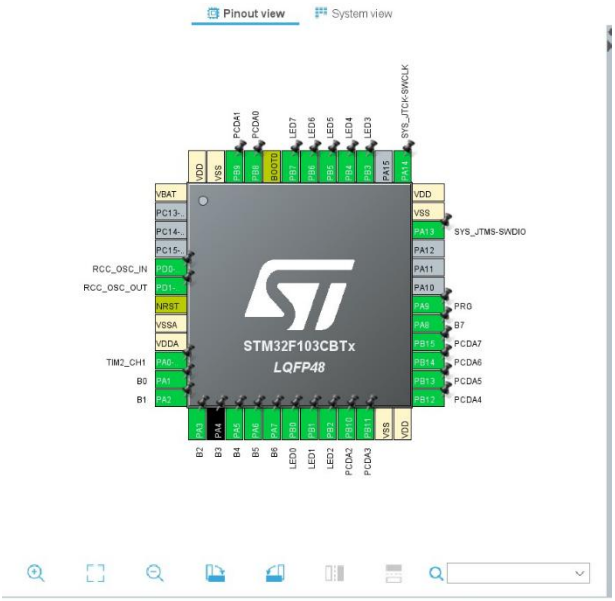


Schema bloc din figura de mai sus prezintă arhitectura internă a microcontrolerului STM32F103. Principalele componente sunt:

- **Cortex-M3 CPU** – nucleul procesorului, cu frecvență maximă de 48/72 MHz, conectat la memorie prin bus-urile Ibus și Dbus
- **Memorie** – Flash 512 KB pentru stocarea programului și SRAM 64 KB pentru date
- **NVIC** – controlerul de întreruperi, gestionează toate întreruperile sistemului
- **Bus Matrix** – interconectează CPU cu perifericele prin magistralele AHB, APB1 și APB2
- **GPIO** – porturi de intrare/ieșire (PA–PG), folosite pentru butoane, LED-uri și comunicația cu DSP-ul
- **Timere** – TIM1–TIM8, dintre care noi folosim TIM2 pentru generarea întreruperii periodice
- **Alimentare** – tensiune de lucru 3.3V–1.8V, cu supervizare PVD și oscilator intern RC 8 MHz / extern XTAL 4–16 MHz

Schema de configurare a pinilor (STM32CubeIDE – Pinout View)

În figura de mai jos este prezentată configurarea pinilor microcontrolerului STM32F103CBTx realizată în STM32CubeIDE.



Pinii au fost configurați astfel:

Intrări (GPIOA): PA1–PA8 sunt configurați ca intrări digitale (Input mode, fără pull-up/pull-down) și corespund butoanelor B0–B7. PA9 este de asemenea intrare și corespunde butonului PRG.

Ieșiri (GPIOB): PB0–PB7 sunt configurați ca ieșiri push-pull (Output Push Pull, nivel inițial Low) și controlează LED-urile LED0–LED7 pentru afișarea stării interne a sistemului ARM. PB8–PB15 sunt tot ieșiri push-pull și formează portul de comandă PCDA0–PCDA7, prin care se transmite comanda către DSP.

Oscilator (GPIOD): PD0 și PD1 sunt rezervați pentru oscilatorul extern (RCC_OSC_IN și RCC_OSC_OUT), folosit ca sursă de ceas pentru PLL.

Configuration

Group By Peripherals

GPIO | RCC | SYS | TIM

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull-down	Maximum output ...	User Label	Modified
PA1	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B0	☑
PA2	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B1	☑
PA3	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B2	☑
PA4	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B3	☑
PA5	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B4	☑
PA6	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B5	☑
PA7	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B6	☑
PA8	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	B7	☑
PA9	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	PRG	☑

PB0	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED0	☑
PB1	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED1	☑
PB2	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED2	☑
PB3	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED3	☑
PB4	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED4	☑
PB5	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED5	☑
PB6	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED6	☑
PB7	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED7	☑
PB8	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA0	☑
PB9	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA1	☑
PB10	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA2	☑

PB5	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED5	☑
PB6	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED6	☑
PB7	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED7	☑
PB8	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA0	☑
PB9	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA1	☑
PB10	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA2	☑
PB11	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA3	☑
PB12	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA4	☑
PB13	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA5	☑
PB14	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA6	☑
PB15	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	PCDA7	☑

Configuration

Group By Peripherals

GPIO | RCC | SYS | TIM

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull-...	Maximum output s...	User Label	Modified
PD0-OSC_IN	RCC_OSC_IN	n/a	n/a	n/a	n/a		☐
PD1-OSC_OUT	RCC_OSC_OUT	n/a	n/a	n/a	n/a		☐

5. DSP

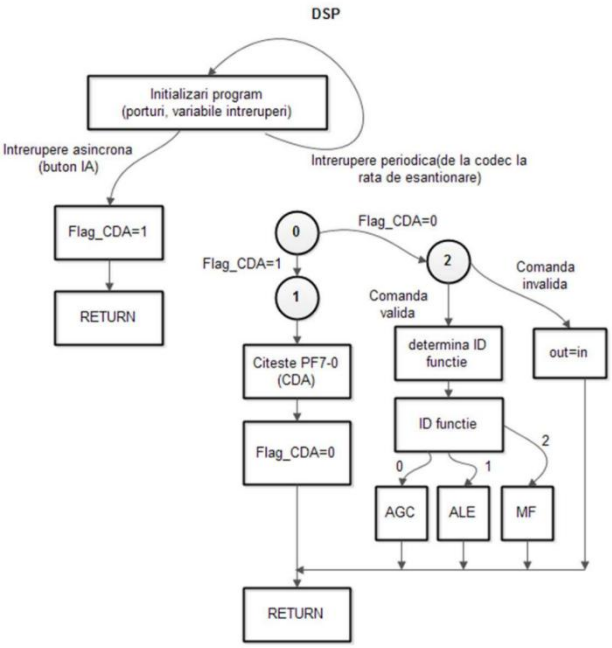


Figura 4. Funcțiile implementate pe DSP

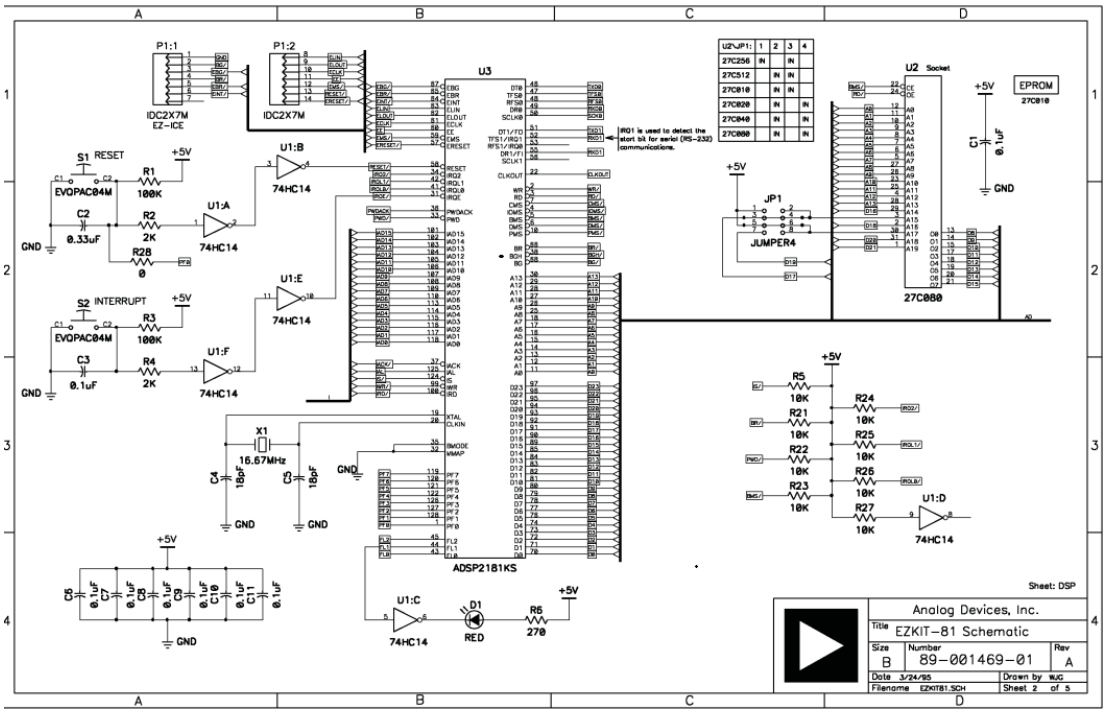
DSP-ul are rolul de a procesa semnalul primit de la CODEC și de a aplica funcția selectată de ARM. Programul DSP este condus de două surse de întreruperi, vizibile în organigrama din Figura 4.

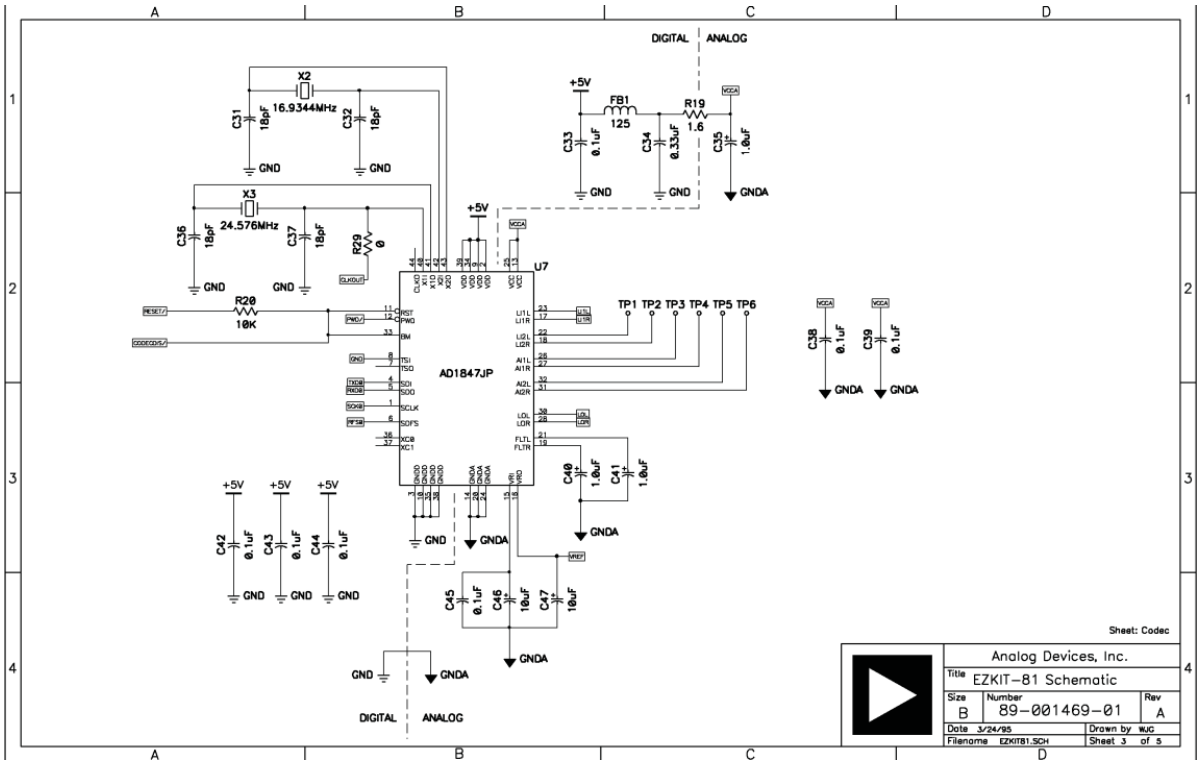
Întreruperea asincronă (buton IA): Când utilizatorul apasă butonul IA, se generează o întrerupere asincronă. Handler-ul acesteia face un singur lucru: setează Flag_CDA = 1 și revine (RETURN). Practic acest flag semnalizează că ARM-ul a pus o comandă nouă pe portul PF7-0.

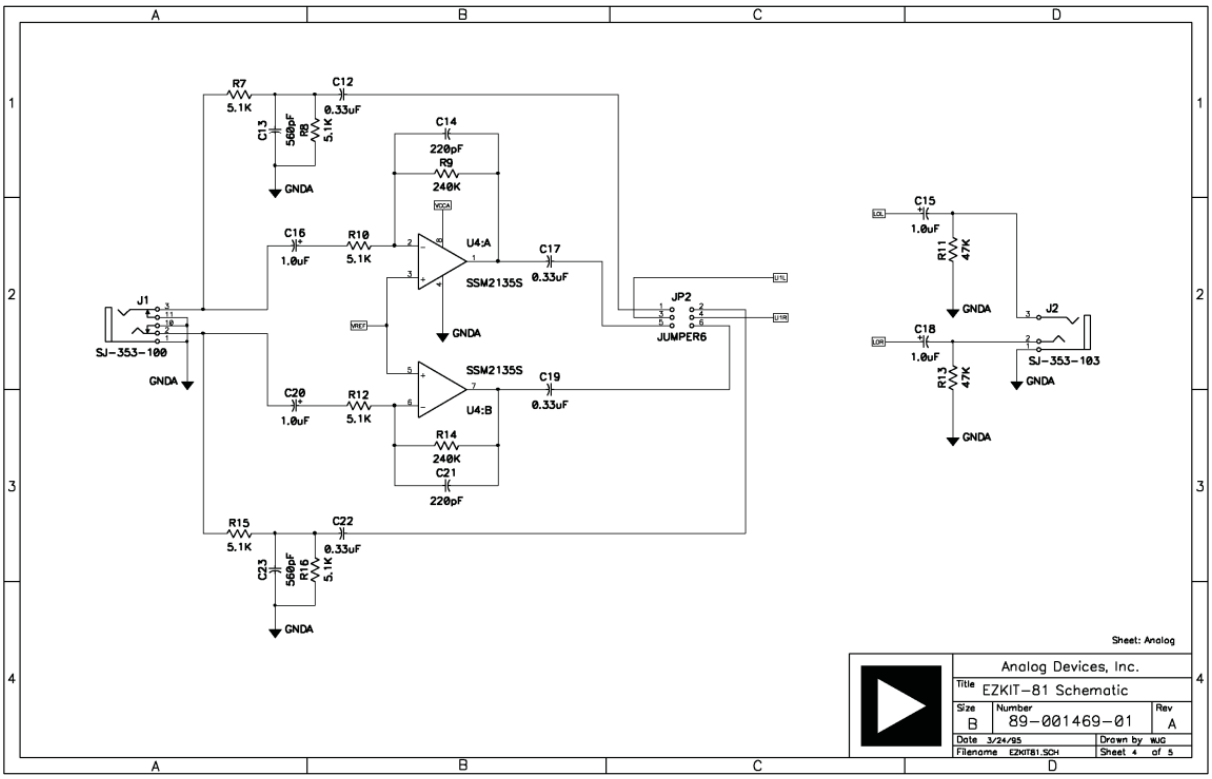
Întreruperea periodică (de la CODEC, la rata de eșantionare): Aceasta este întreruperea principală și implementează mașina de stări cu 3 stări:

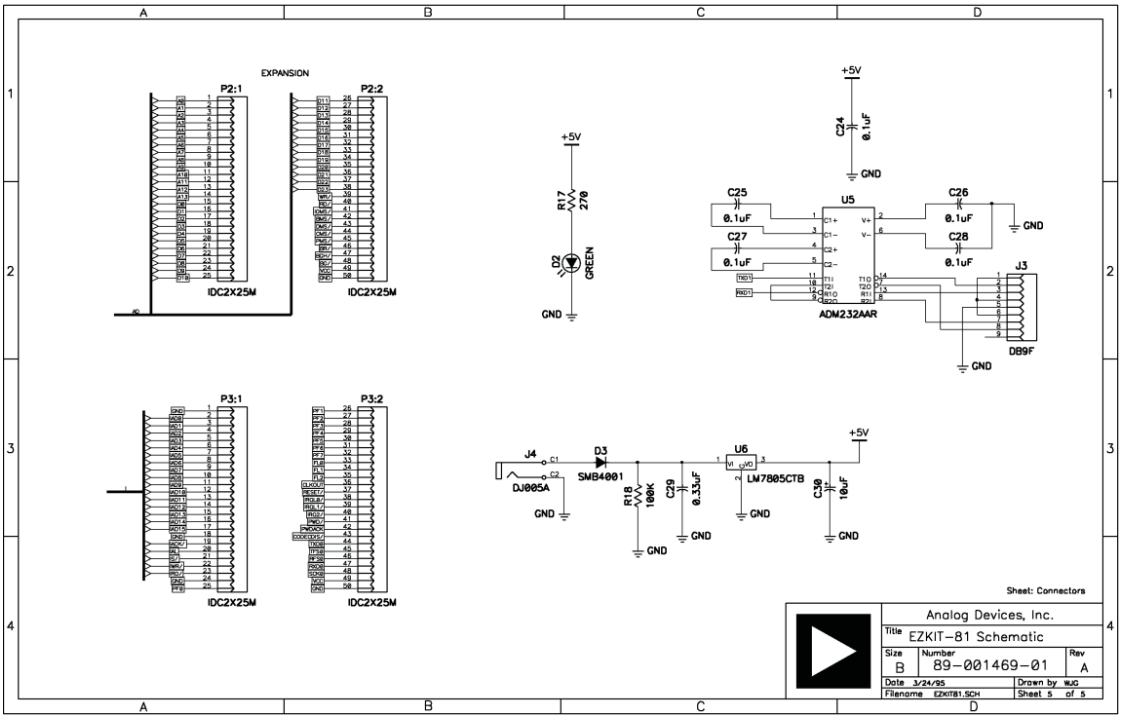
- **Starea 0** – verifică Flag_CDA. Dacă Flag_CDA = 1 înseamnă că există o comandă nouă → trece în Starea 1. Dacă Flag_CDA = 0 → trece direct în Starea 2.
- **Starea 1** – citește comanda de pe portul PF7-0 (CDA), resetează Flag_CDA = 0 și revine (RETURN). Comanda va fi procesată abia la următoarea întrerupere în Starea 2.
- **Starea 2** – verifică dacă comanda este validă (D7=D6=1). Dacă este validă, determină ID-ul funcției și apelează AGC, ALE sau MF corespunzător. Dacă comanda este invalidă → bypass (out = in). La final face RETURN.

Schema electrică DSP

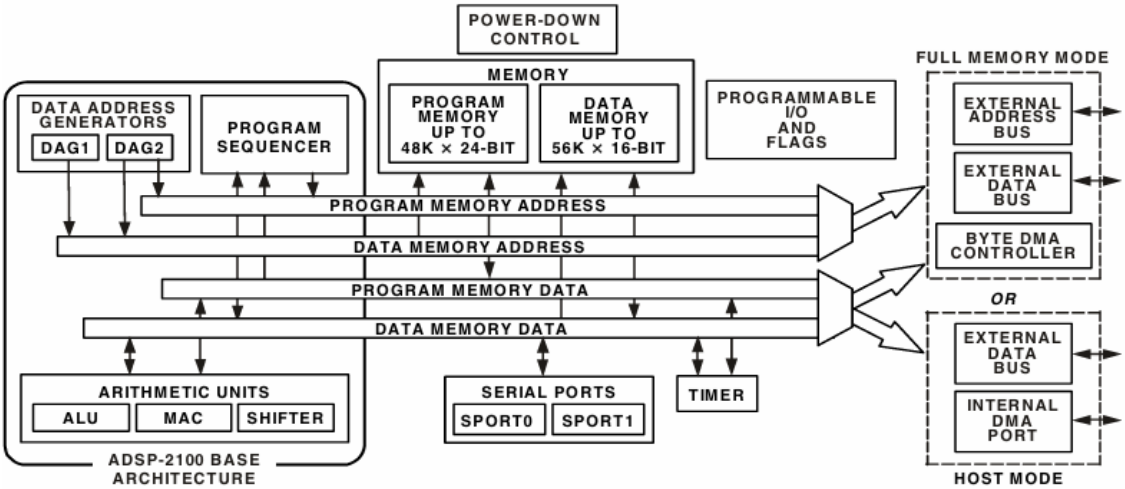



DIGITAL
ANALOG
X2
16.9344MHz
C31 18pF
C32 18pF
GND
X3
24.576MHz
C36 18pF
C37 18pF
R29 0
CLKOUT
+5V
FB1 125
R19 1.6
C33 0.1uF
C34 0.33uF
C35 1.0uF
VCCA
GNDA
U7
AD1847JP
R20 10K
RESET/
PWD/
XOUT/CCLK/
TP1 TP2 TP3 TP4 TP5 TP6
C38 0.1uF
C39 0.1uF
C40 1.0uF
C41 1.0uF
C42 0.1uF
C43 0.1uF
C44 0.1uF
C45 0.1uF
C46 10uF
C47 10uF
VREF
Sheet: Codec
Analog Devices, Inc.
Title EZKIT-81 Schematic
Size B
Number 89-001469-01
Rev A
Date 3/24/95
Drawn by WJC
Filename EZKIT81.SCH
Sheet 3 of 5


R7 5.1K
C12 0.33uF
C13 560pF
R8 5.1K
C14 220pF
R9 240K
VCCA
C16 1.0uF
R10 5.1K
U4:A
SSM2135S
C17 0.33uF
J1
SJ-353-100
GNDA
VREF
JP2
JUMPER6
U7L
U7R
C20 1.0uF
R12 5.1K
SSM2135S
U4:B
C19 0.33uF
R14 240K
C21 220pF
R15 5.1K
C22 0.33uF
C23 560pF
R16 5.1K
C15 1.0uF
R11 47K
LOL
J2
C18 1.0uF
R13 47K
LOR
SJ-353-103
Sheet: Analog
Analog Devices, Inc.
Title EZKIT-81 Schematic
Size B
Number 89-001469-01
Rev A
Date 3/24/95
Drawn by WJC
Filename EZKIT81.SCH
Sheet 4 of 5



Schema bloc ADSP2181



Arhitectura internă se bazează pe structura ADSP-2100 și conține:

- **Unități aritmetice** – ALU, MAC (multiplicator-acumulator) și Shifter, folosite pentru calculele din funcțiile AGC, ALE și MF
- **Program Sequencer** – controlează execuția instrucțiunilor
- **DAG1, DAG2** – generatoare de adrese pentru accesul eficient la bufferele circulare (folosite în codul nostru pentru rx_buf și tx_buf)

- **Memorie Program** – până la 48K × 24-bit (secțiunile pm_da, interrupts, seg_code din codul nostru)
- **Memorie Date** – până la 56K × 16-bit (secțiunile buf_var1/2/3, data1 din codul nostru)
- **SPORT0, SPORT1** – porturi seriale; noi folosim SPORT0 pentru comunicația cu CODEC-ul AD1847
- **Programmable I/O and Flags** – pinii PF0–PF7, folosiți pentru citirea comenzii de la ARM

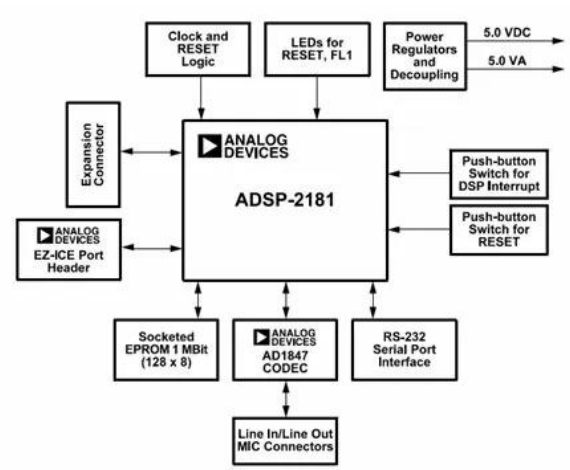
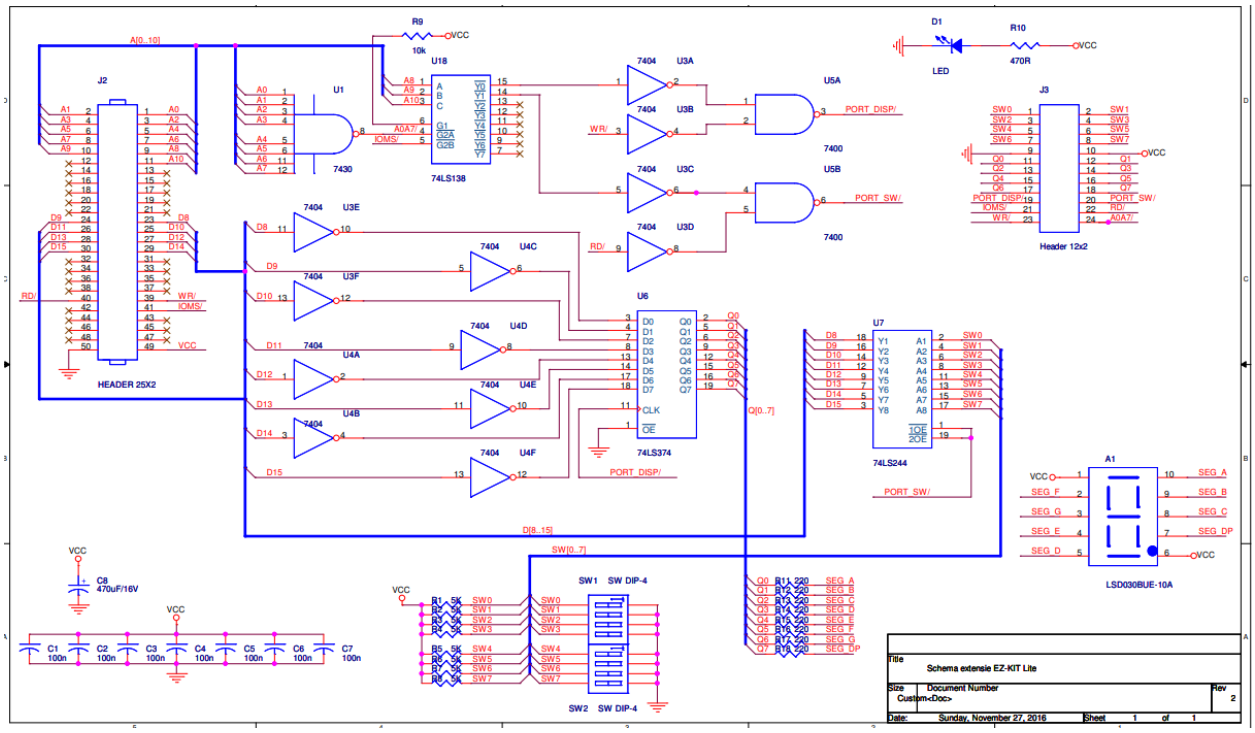


Figure 3-1. ADSP-2181 EZ-KIT Lite System Block Diagram

Extensia I/O DSP



Schema de mai sus reprezintă extensia I/O a plăcii EZ-KIT Lite ADSP-2181. Aceasta asigură interfața dintre DSP și utilizator prin comutatoare și afișor. Principalele componente sunt:

- **HEADER 25x2 (J2)** – conectorul principal prin care extensia se leagă la placa EZ-KIT Lite, aducând bus-ul de date și semnalele de control (RD/, WR/, IOMS/)
- **7430** – poartă NAND cu 8 intrări, folosită pentru decodarea adreselor
- **74LS138 (U18)** – decodor 3-la-8, selectează unul dintre porturile de I/O în funcție de adresă
- **7404** – inversoare logice, folosite pentru adaptarea semnalelor de control (RD/, WR/, PORT_DISP/, PORT_SW/)
- **7400** – porți NAND cu 2 intrări, logică suplimentară de selecție porturi
- **74LS374 (U6)** – registru octet cu latch, reține valoarea scrisă de DSP pentru afișare pe segmente
- **74LS244 (U7)** – buffer pentru citirea stării comutatoarelor SW0–SW7
- **SW1, SW2 (DIP-4)** – comutatoare pentru setarea parametrilor funcțiilor DSP (K pentru MF, etc.)
- **J3 (Header 12x2)** – conector pentru comutatoarele SW0–SW7
- **Afișor cu 7 segmente (A1 – LSD030BUE-10A)** – afișează starea internă a programului DSP (funcția activă)
- **LED (D1)** + rezistență 470Ω – indicator vizual de stare
- **Condensatori de decuplare** (C1–C8) – filtrare alimentare

6. Cod – STM32

```
/* USER CODE BEGIN Header */
/**
 ******************************************************************************
 * @file           : main.c
 * @brief          : Main program body
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
```

```
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
TIM_HandleTypeDef htim2;

/* USER CODE BEGIN PV */
uint8_t comanda = 0;        // comanda care se trimite la DSP
uint8_t trs_cda = 0;        // flag pentru butonul PRG
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM2_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2)
  {
    GPIO_PinState prg = HAL_GPIO_ReadPin(GPIOA, PRG_Pin);

    switch (trs_cda)
    {
      case 0:
        if (prg == GPIO_PIN_RESET)
        {
            trs_cda = 1;
        }
        break;

      case 1:
```

```
      if (prg == GPIO_PIN_SET)
      {
        uint16_t stare_port_A = GPIOA->IDR;
        uint8_t date_butoane = (uint8_t)((stare_port_A >> 1) & 0x3F);

        comanda = 0xC0 | date_butoane;

        trs_cda = 2;
      }
      break;

    case 2:
      GPIOB->ODR = ((uint16_t)comanda << 8) | (uint16_t)comanda;

      trs_cda = 0;
      break;

    default:
            trs_cda = 0;
      break;
  }
 }
}
/* USER CODE END 0 */

/**
 * @brief  The application entry point.
 * @retval int
 */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_TIM2_Init();
  /* USER CODE BEGIN 2 */
  HAL_TIM_Base_Start_IT(&htim2);
  /* USER CODE END 2 */

  /* Infinite loop */
```

```
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief TIM2 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM2_Init(void)
{

  /* USER CODE BEGIN TIM2_Init 0 */

  /* USER CODE END TIM2_Init 0 */
```

```
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};

  /* USER CODE BEGIN TIM2_Init 1 */

  /* USER CODE END TIM2_Init 1 */
  htim2.Instance = TIM2;
  htim2.Init.Prescaler = 7199;
  htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim2.Init.Period = 100;
  htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_OC_Init(&htim2) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_TIMING;
  sConfigOC.Pulse = 0;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_OC_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM2_Init 2 */

  /* USER CODE END TIM2_Init 2 */
  HAL_TIM_MspPostInit(&htim2);

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
/* USER CODE BEGIN MX_GPIO_Init_1 */
/* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
```

```
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOB, LED0_Pin|LED1_Pin|LED2_Pin|PCDA2_Pin
                    |PCDA3_Pin|PCDA4_Pin|PCDA5_Pin|PCDA6_Pin
                    |PCDA7_Pin|LED3_Pin|LED4_Pin|LED5_Pin
                    |LED6_Pin|LED7_Pin|PCDA0_Pin|PCDA1_Pin, GPIO_PIN_RESET);

  /*Configure GPIO pins : B0_Pin B1_Pin B2_Pin B3_Pin
                     B4_Pin B5_Pin B6_Pin B7_Pin
                     PRG_Pin */
  GPIO_InitStruct.Pin = B0_Pin|B1_Pin|B2_Pin|B3_Pin
                    |B4_Pin|B5_Pin|B6_Pin|B7_Pin
                    |PRG_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : LED0_Pin LED1_Pin LED2_Pin PCDA2_Pin
                     PCDA3_Pin PCDA4_Pin PCDA5_Pin PCDA6_Pin
                     PCDA7_Pin LED3_Pin LED4_Pin LED5_Pin
                     LED6_Pin LED7_Pin PCDA0_Pin PCDA1_Pin */
  GPIO_InitStruct.Pin = LED0_Pin|LED1_Pin|LED2_Pin|PCDA2_Pin
                    |PCDA3_Pin|PCDA4_Pin|PCDA5_Pin|PCDA6_Pin
                    |PCDA7_Pin|LED3_Pin|LED4_Pin|LED5_Pin
                    |LED6_Pin|LED7_Pin|PCDA0_Pin|PCDA1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/* USER CODE BEGIN MX_GPIO_Init_2 */
/* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
```

```
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

7. Cod – VisualDSP++

```
#include "def2181.h"

#define PORT_OUT 0xFF
#define PORT_IN  0x1FF


.SECTION/DM     buf_var1;
.var    rx_buf[3];

.SECTION/DM     buf_var2;
.var    tx_buf[3] = 0xc000, 0x0000, 0x0000;
.SECTION/DM     buf_var3;
.var    init_cmds[13] =
        0xc002,
        0xc102,
        0xc288,
        0xc388,
        0xc488,
        0xc588,
        0xc680,
        0xc780,
        0xc85c,
        0xc909,
        0xca00,
        0xcc40,
        0xcd00;


.SECTION/DM     data1;
```

```
.var    stat_flag;

.var    PF_input;
.var    PF_output;

.var    Flag_CDA;
.var    cmd_comanda;
.var    cmd_corect;
.var    id_functie;
.var    x_in;
.var    y_out;



.SECTION/PM     pm_da;


/*intreruperi*/

.SECTION/PM     interrupts;

        jump start;             rti; rti; rti;     /* 00: reset            */

        rti;                    rti; rti; rti;     /* 04: IRQ2             */

        rti;                    rti; rti; rti;     /* 08: IRQL1            */

        rti;                    rti; rti; rti;     /* 0C: IRQL0            */

        ar = dm(stat_flag);                        /* 10: SPORT0 tx        */
        ar = pass ar;
        if eq rti;
        jump next_cmd;

        jump input_samples;                        /* 14: SPORT0 rx        */
        rti;                    rti; rti;

        jump isr_ia;            rti; rti; rti;     /* 18: IRQE  */

        rti;                    rti; rti; rti;     /* 1C: BDMA             */
        rti;                    rti; rti; rti;     /* 20: SPORT1 tx or IRQ1 */
        rti;                    rti; rti; rti;     /* 24: SPORT1 rx or IRQ0 */
```

```
        nop;                    rti; rti; rti;     /* 28: timer            */
        rti;                    rti; rti; rti;     /* 2C: power down       */


.SECTION/PM     seg_code;

start:
        /* shut down sport 0 */
        ax0 = b#0000100000000000;
        dm(Sys_Ctrl_Reg) = ax0;

        ena timer;

        i5 = rx_buf;
        l5 = LENGTH(rx_buf);
        i6 = tx_buf;
        l6 = LENGTH(tx_buf);
        i3 = init_cmds;
        l3 = LENGTH(init_cmds);

        m1 = 1;
        m5 = 1;

        ax0 = 0;
        dm(Flag_CDA)    = ax0;
        dm(cmd_comanda) = ax0;
        dm(cmd_corect)  = ax0;
        dm(id_functie)  = ax0;
        dm(x_in)        = ax0;
        dm(y_out)       = ax0;


/* SPORT0 CONFIG*/
        ax0 = b#0000110011010111;
        dm(Sport0_Autobuf_Ctrl) = ax0;

        ax0 = 0;
        dm(Sport0_Rfsdiv) = ax0;

        ax0 = 0;
        dm(Sport0_Sclkdiv) = ax0;

        ax0 = b#1000011000001111;
```

```
        dm(Sport0_Ctrl_Reg) = ax0;

        ax0 = b#0000000000000111;
        dm(Sport0_Tx_Words0) = ax0;

        ax0 = b#0000000000000111;
        dm(Sport0_Tx_Words1) = ax0;

        ax0 = b#0000000000000111;
        dm(Sport0_Rx_Words0) = ax0;

        ax0 = b#0000000000000111;
        dm(Sport0_Rx_Words1) = ax0;


/* SYSTEM CONFIG*/
        ax0 = b#0001100000000000;
        dm(Sys_Ctrl_Reg) = ax0;

        ifc = b#00000011111110;
        nop;

        icntl = b#00010;

        mstat = b#1100000;

 /******************************
   ADSP 1847 Codec intialization
 ******************************/

        ax0 = 1;
        dm(stat_flag) = ax0;

        ena ints;
        imask = b#0001010001;

        ax0 = dm(i6, m5);
        tx0 = ax0;

check_init:
        ax0 = dm(stat_flag);
        af = pass ax0;
        if ne jump check_init;
```

```
        ay0 = 2;
check_aci1:
        ax0 = dm(rx_buf);
        ar = ax0 and ay0;
        if eq jump check_aci1;

check_aci2:
        ax0 = dm(rx_buf);
        ar = ax0 and ay0;
        if ne jump check_aci2;
        idle;

        ay0 = 0xbf3f;
        ax0 = dm(init_cmds + 6);
        ar = ax0 and ay0;
        dm(tx_buf) = ar;
        idle;

        ax0 = dm(init_cmds + 7);
        ar = ax0 and ay0;
        dm(tx_buf) = ar;
        idle;

        ifc = b#00000011111110;
        nop;

        imask = b#0001100001;

// wait states
        si = 0xFFFF;
        dm(Dm_Wait_Reg) = si;

// PF ports
        si = 0x0000;
        dm(Prog_Flag_Comp_Sel_Ctrl) = si;

wt:
                idle;
        jump wt;


isr_ia:
```

```
        ena sec_reg;
        ax0 = 1;
        dm(Flag_CDA) = ax0;
        rti;

input_samples:
        ena sec_reg;

        mr1 = dm(rx_buf + 1);

        dm(x_in) = mr1;

        ax0 = dm(Flag_CDA);
        ar = pass ax0;
        if ne jump starea_1;
        jump starea_2;


starea_1:
         ax0 = dm(Prog_Flag_Data);
        dm(cmd_comanda) = ax0;
        dm(PF_input) = ax0;

        ax0 = 0;
        dm(Flag_CDA) = ax0;

        rti;


starea_2:
        ax0 = dm(cmd_comanda);
        ay0 = 0x00C0;
        ar = ax0 and ay0;
        af = ar - ay0;
        if ne jump cmd_invalida;

        ax0 = 1;
        dm(cmd_corect) = ax0;

        ax0 = dm(cmd_comanda);
        ay0 = 0x0007;
        ar = ax0 and ay0;
        dm(id_functie) = ar;
```

```
        ax0 = dm(id_functie);

        ay0 = 0;
        ar = ax0 - ay0;
        if eq jump agc_func;

        ay0 = 1;
        ar = ax0 - ay0;
        if eq jump ale_func;

        ay0 = 2;
        ar = ax0 - ay0;
        if eq jump mf_func;

        jump bypass;

cmd_invalida:
        ax0 = 0;
        dm(cmd_corect) = ax0;
        jump bypass;

//cele 3 functii de mai jos se vor implementa in etapa a II-a

agc_func:
        jump bypass;

ale_func:
        jump bypass;

mf_func:
        jump bypass;


bypass:
        mr1 = dm(x_in);
        dm(y_out) = mr1;
        jump output;

output:
        dm(tx_buf + 1) = mr1;       /* canal stanga */
        dm(tx_buf + 2) = mr1;       /* canal dreapta */
        rti;
```

```
next_cmd:
        ena sec_reg;
        ax0 = dm(i3, m1);
        dm(tx_buf) = ax0;

        ax0 = i3;
        ay0 = init_cmds;
        ar = ax0 - ay0;
        if gt rti;

        ax0 = 0xaf00;
        dm(tx_buf) = ax0;
        ax0 = 0;
        dm(stat_flag) = ax0;
        rti;
```